

Rational Agents and Problem Solving as Search

From PEAS specifications to executable state-space models

Learning objectives

By the end of this lesson, you should be able to:

- 01 — Define an agent, agent function, rational action, percept, sensor, and actuator.
- 02 — Write a complete PEAS specification for a nontrivial task environment.
- 03 — Classify the environment properties that constrain algorithm choice.
- 04 — Compare simple reflex, model-based, goal-based, and utility-based agents.
- 05 — Formalize a domain as a five-component search problem and Python interface.
- 06 — Distinguish search trees from search graphs and compare exploration policies.

An agent maps percept history to action

$$f : P^* \rightarrow A \quad a_t = f(\langle p_1, p_2, \dots, p_t \rangle)$$

- P^* is the set of finite percept sequences, not merely the current sensor reading.
- A is the set of actions available through the agent's actuators.
- An agent program implements f on a particular machine architecture.

ENGINEERING IMPLICATION

Rational means selecting an action expected to maximize the stated performance measure, given available evidence and knowledge.

PEAS specifies the task before the implementation

01

Performance

Define observable success, priorities, penalties, and tradeoffs.

02

Environment

Name relevant entities, dynamics, constraints, and other agents.

03

Actuators

List every channel through which the agent can change the world.

04

Sensors

List observations, resolution, latency, noise, and missing data.

PEAS is a requirements artifact: it defines the decision problem before code narrows the design.

Worked PEAS: campus delivery robot, part I

Performance measure

- Deliver the correct parcel to the authorized recipient.
- Avoid collisions and respect pedestrian right-of-way.
- Meet delivery windows while limiting energy and route length.
- Prefer safe stop and escalation when confidence is low.

Environment

- Sidewalks, crossings, ramps, doors, elevators, and charging stations.
- Pedestrians, cyclists, vehicles, animals, weather, and construction.
- Dispatch service, map service, recipients, and human supervisors.
- Changing access rules, temporary obstacles, and wireless outages.

Worked PEAS: campus delivery robot, part II

Actuators

- Drive, steer, brake, and hold position.
- Signal with lights, sound, and a display.
- Open or lock the parcel compartment.
- Request assistance, replan, or enter a safe-stop mode.

Sensors

- Cameras, depth sensing, wheel odometry, inertial sensing, and GNSS.
- Battery, motor, compartment, and bumper telemetry.
- Digital maps, traffic controls, dispatch messages, and access credentials.
- Recipient authentication and supervisor commands.

Environment properties determine what memory is needed

Observability

What does the percept reveal?

- Fully observable: relevant state is available.
- Partially observable: state must be estimated.
- Sensor noise can make a nominally visible variable uncertain.

Dynamics

How predictable are transitions?

- Deterministic: action and state fix the successor.
- Stochastic: transitions have outcome probabilities.
- Strategic: another agent selects consequential actions.

Time

Can the world change while deciding?

- Static: world waits during deliberation.
- Dynamic: state may change before action.
- Semidynamic: world fixed, but performance changes with time.

Environment properties also shape the objective

Temporal coupling

Does one decision affect the next?

- Episodic tasks contain independent decisions.
- Sequential tasks make present actions alter future options.
- Credit assignment matters when consequences are delayed.

Representation

Are states and actions countable?

- Discrete models enumerate symbols or finite choices.
- Continuous models use real-valued state or control.
- A software abstraction may discretize a continuous world.

Participants

Whose choices affect performance?

- Single-agent models treat others as environment dynamics.
- Cooperative agents share objectives.
- Competitive agents create adversarial choices.

Checkpoint: classify the robot's environment

CHECKPOINT

A robot sees only nearby corridors, motion is noisy, pedestrians move during planning, and deliveries affect its remaining battery. How should the task be classified?

01

Name the observability and transition properties.

02

Name the time and temporal-coupling properties.

03

Identify one modeling choice that could change a classification.

Agent architectures add information and decision power

Simple reflex

Current percept -> rule

- Condition-action rules only.
- No internal history or world state.
- Fast, but brittle under hidden state.

Model-based reflex

Percept + model -> state -> rule

- Maintains an internal state estimate.
- Uses transition and sensor models.
- Handles some partial observability.

Goal-based

State + goal -> plan

- Tests whether candidate states satisfy a goal.
- Searches or plans over future consequences.
- Needs a transition model and goal test.

Utility resolves tradeoffs that a goal test cannot

Goal-based agent

- Classifies states as goal or non-goal.
- Any goal-reaching plan may be acceptable.
- Path cost can prefer one solution when explicitly modeled.
- Does not generally rank uncertain outcome distributions.

Utility-based agent

- Assigns utility to outcomes or histories.
- Ranks speed, safety, energy, comfort, or other tradeoffs.
- Chooses maximum expected utility under uncertainty.
- Requires calibrated preferences, not merely a destination.

Checkpoint: choose an agent architecture

CHECKPOINT

A thermostat observes current temperature, estimates room heat loss, and chooses actions to minimize energy while maintaining a comfort range. Which agent type is the best description?

01

What internal state or model is required?

02

Why is a simple reflex label incomplete?

03

What does utility add beyond a Boolean comfort goal?

A search problem has five operational components

Problem = $\langle s_0, \text{ACTIONS}, \text{RESULT}, \text{GOAL-TEST}, c \rangle$

- s_0 in S is the initial state.
- $\text{ACTIONS}(s)$ returns the legal actions available in state s .
- $\text{RESULT}(s, a) \rightarrow s'$ defines the successor; $\text{GOAL-TEST}(s) \rightarrow \text{Boolean}$.
- $c(s, a, s') > 0$ is step cost; path cost g is the sum over a path.

ENGINEERING IMPLICATION

The state representation must contain enough information to predict legal successors, goal status, and future cost.

Worked formalization: blocked gridworld

SCENARIO

A 3 by 4 grid starts at (0,0), targets (2,3), blocks (1,1), and permits four unit moves.

- S = all in-bounds unblocked (row, column) tuples; $s_0 = (0,0)$.
- $ACTIONS(s)$ = moves whose destinations remain in S .
- $RESULT((r,c), move)$ adds the move's row and column delta.
- $GOAL-TEST(s)$ is $s = (2,3)$; every legal step has cost 1.

One legal solution is Right, Right, Right, Down, Down with path cost $g = 5$.

The formal tuple becomes a Problem interface

Mathematical component

- Initial state s_0
- $ACTIONS(s)$ and $RESULT(s,a)$
- $GOAL-TEST(s)$
- Step cost $c(s,a,s')$

Executable contract

- `initial_state()` returns an immutable, validated State.
- `actions(state)` is deterministic; result rejects illegal actions.
- `goal_test(state)` returns a real Boolean after validation.
- `step_cost` validates the transition and returns a positive number.

Search trees represent paths; search graphs deduplicate states

Search tree

A node identifies a path

- Root is the initial-state node.
- Children extend one path by one action.
- The same state may appear in many nodes.
- Cycles can produce infinite depth.

Search graph

States are recognized across paths

- Equal states are deduplicated.
- An explored set records processed states.
- Frontier membership may prevent duplicate work.
- Cheaper rediscovery may require an update.

Engineering effect

Equality controls performance

- Stable hashing enables near-constant membership.
- Mutable state keys corrupt deduplication.
- Node identity is not state identity.
- Memory trades against repeated expansion.

Generic graph search separates control from ordering

```
function GRAPH_SEARCH(problem, frontier):
    frontier.add(node(problem.initial_state))
    explored = empty set

    while not frontier.empty():
        node = frontier.remove()
        if problem.goal_test(node.state):
            return reconstruct(node)

        if node.state in explored:
            continue
        explored.add(node.state)

        for action in problem.actions(node.state):
            child = make_child(problem, node, action)
            frontier.add_or_update(child)

    return failure
```

Stable abstractions

- Problem defines the state space.
- Node records one path.
- Frontier defines removal priority.
- Explored prevents repeated expansion.

Frontier policy is the unifying thread of Module A

Depth or breadth

Order by structural position

- BFS removes minimum depth first.
- DFS removes the most recently added path.
- Completeness and memory differ sharply.

Cost or estimate

Order by numeric priority

- UCS orders by $g(n)$.
- Greedy orders by $h(n)$.
- A-star orders by $g(n) + h(n)$.

Adversarial choice

Back up values through a game tree

- Minimax alternates maximizing and minimizing.
- Alpha-beta changes expansion, not minimax value.
- Expectimax averages chance outcomes.

Checkpoint: repair an incomplete formalization

CHECKPOINT

A route state stores only location, but toll price depends on time and a battery constraint depends on remaining charge. Is location alone a valid state?

01

Name the missing variables.

02

Explain which search functions become inconsistent.

03

Propose a hashable state representation.

Key terms: behavior and environment

Agent vocabulary

- Percept: one sensor input; percept sequence: observation history.
- Agent function: mapping from percept histories to actions.
- Rational action: maximizes expected performance under available evidence.
- Utility: numeric preference over outcomes or histories.

Environment vocabulary

- Observable: whether relevant state is available.
- Deterministic or stochastic: how actions determine outcomes.
- Episodic or sequential: whether decisions affect later decisions.
- Static or dynamic: whether the world changes during deliberation.

Key takeaways

- 01 — Specify performance and interfaces with PEAS before selecting an algorithm.
- 02 — Rationality is expected-performance maximization under available evidence, not omniscience.
- 03 — A search state must be sufficient for actions, transitions, goals, and costs.
- 04 — Search algorithms share a state-space abstraction but differ in ordering, backup, and duplicate policy.

Next: Week 2 begins with BFS, DFS, uniform-cost search, and their completeness, optimality, time, and space tradeoffs.

Check your understanding

Q1

Why can a rational action lead to a poor outcome without contradicting rationality?

Q2

Write a PEAS specification for an automated code-review agent.

Q3

Classify online chess by observability, determinism, temporal coupling, dynamics, representation, and participants.

Q4

Formalize a two-room vacuum world as $\langle s_0, \text{ACTIONS}, \text{RESULT}, \text{GOAL-TEST}, c \rangle$.

Q5

Why can a finite graph induce an infinite search tree, and what makes an explored set safe?